GPU Run Agent — MVP Scope

Nov 3, 2026 · @Ved Sharma

Goal

Build an MCP server that takes a plain-English GPU job request, picks the cheapest provider and GPU that can actually run it, launches it, watches it, and tears it down, with every run's cost logged. Target: a working demo after six milestones, built at whatever pace the prototype allows.

The headline demo, run from Claude Code:

> "Fine-tune Qwen2.5-1.5B with LoRA on this dataset, keep it under $5."

The agent estimates memory and runtime, quotes Modal, RunPod and Daytona, shows the plan for approval, launches the winner, streams logs, saves the adapter, and reports estimated vs. actual cost.

What this proves to a reviewer: agent tool design, real ML infrastructure judgment (memory estimation, spot vs. on-demand), and safety thinking around agents that spend money.

Scope

The MVP does one job type well on three providers, driven from an existing agent (Claude Code) rather than a custom chat UI.

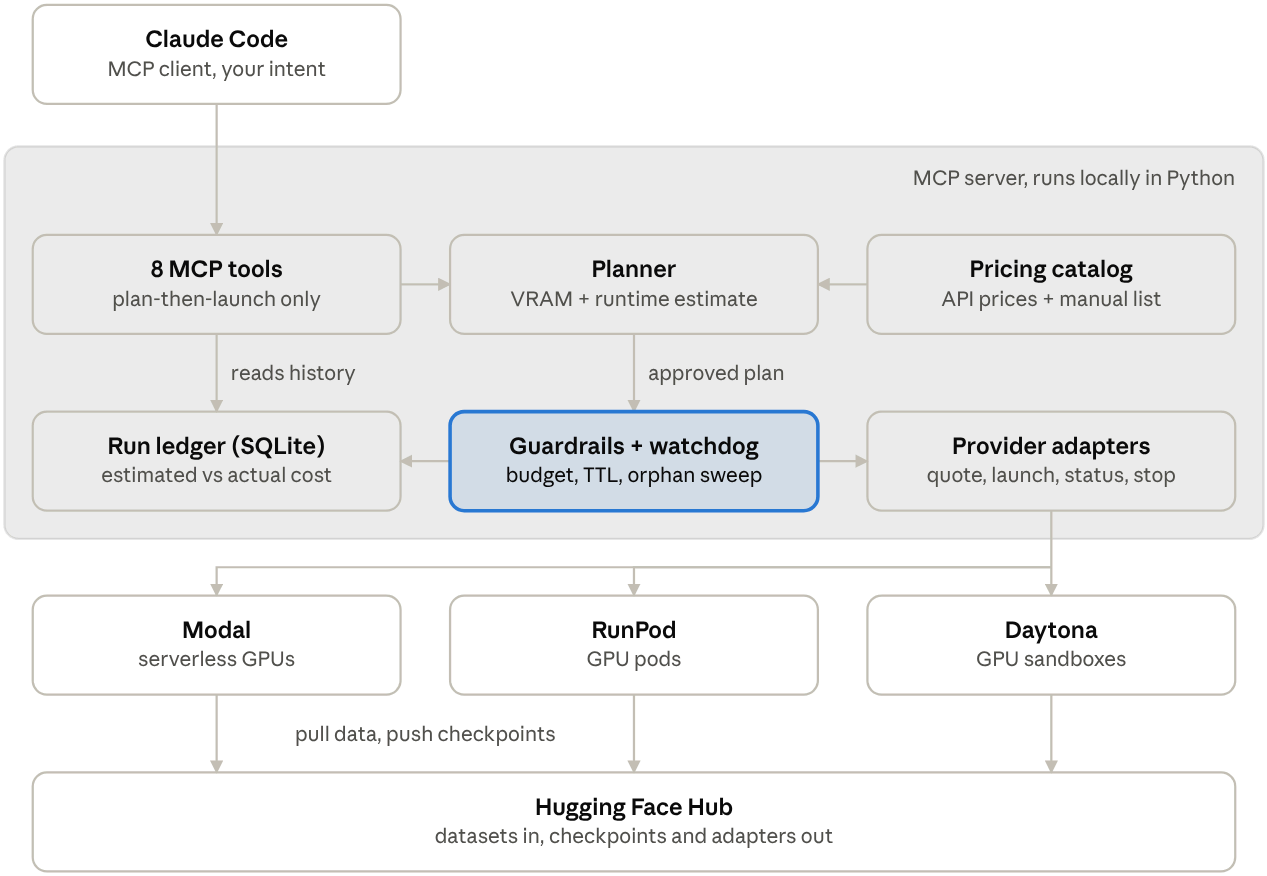
Area	In the MVP	Out (later or never)
Providers	Modal, RunPod, Daytona	Vercel and web deploys, hyperscalers, Kubernetes
Workloads	Single-GPU LoRA/QLoRA fine-tunes, batch eval scripts	Multi-node training, long-lived inference endpoints
Models	Open models up to ~8B params	Anything needing more than one GPU
Interface	MCP server + a small CLI	Web app, chat UI, accounts
Users	Just you, local API keys	Multi-tenant, auth, billing

Area	In the MVP	Out (later or never)
Storage	Datasets and adapters on Hugging Face Hub	Provider-specific volumes as the source of truth

Using the Hub as the neutral data layer keeps every provider interchangeable: each job pulls inputs from the Hub and pushes the adapter back.

Architecture

Every launch passes the guardrails before it reaches a provider



architecture · MCP server, 3 providers, shared Hub storage

Claude Code only sees the tools; the planner decides fit and cost, and nothing reaches a provider without passing the guardrails. Stack: Python, the MCP Python SDK, Pydantic for job specs, SQLite for the ledger, and each provider's own SDK inside its adapter.

MCP tools

Eight tools, split so the agent must plan and quote before it can spend. The server, not the model, owns the decision logic for cost and fit; the model owns intent and the conversation.

Tool	What it does	Spends money
`estimate_job`	Job spec in, VRAM, disk and runtime estimate out (with a confidence range)	No
`get_quotes`	Ranks every provider/GPU combo that fits the estimate by expected total cost	No
`plan_run`	Builds a launch plan with budget cap and TTL; returns a plan ID for approval	No
`launch_run`	Launches an approved plan ID only	Yes
`get_run`	Status, elapsed time, spend so far, last metrics	No
`get_logs`	Tail of the job's logs	No
`stop_run`	Stops a run and confirms teardown on the provider	Ends spend
`list_runs`	Run history with estimated vs. actual cost	No

Every provider implements one adapter interface: `quote`, `launch`, `status`, `logs`, `stop`, `list_active`. Adding a fourth provider means writing one adapter, not touching the agent.

Guardrails

No run can outlive its budget or its TTL, even if the agent or your laptop dies. This is the part reviewers will probe hardest, so it ships in milestone 2, not at the end.

- **Plan-then-launch.** `launch_run` only accepts a plan ID you approved; the model can't invent launch parameters.
- **Hard caps per run.** Max dollars and max wall-clock hours, set in the plan. The watchdog stops the run at either limit.
- **Global caps.** Daily spend limit and max concurrent runs in a local config file the agent can't edit.

- **Provider-side TTL.** Every job also gets the provider's own timeout where one exists, so teardown doesn't depend on your process staying alive.
- **Orphan sweep.** On startup and every few minutes, `list_active` on each provider is reconciled against the ledger; anything unknown is flagged and stopped.
- **Checkpointing.** Training jobs save to the Hub every N steps, so a spot preemption or a cap hit loses minutes, not the run.
- **Local secrets.** API keys stay in your keychain or env; they never pass through the model's context.

Build plan

Six milestones, each ending in something you can run. No fixed dates: move to the next one when the current one works end to end. Keep a hard $50 total cloud budget for development by testing on 0.5B–1.5B models.

1. **Milestone 1: one provider, end to end.** Modal adapter, a containerized LoRA training script (TRL + PEFT), launch/status/logs/stop from a CLI. No agent yet.
2. **Milestone 2: ledger and guardrails.** SQLite run ledger, watchdog with budget and TTL caps, orphan sweep, Hub checkpointing.
3. **Milestone 3: RunPod and Daytona adapters.** Same interface, same training image. Pricing catalog: pull prices from provider APIs where they exist, hand-maintain the rest with a last-updated date.
4. **Milestone 4: estimator and quoting.** VRAM estimate from parameter count, precision, LoRA rank, sequence length and batch size; runtime from tokens × measured throughput per GPU type. `get_quotes` ranks by expected total cost, including startup time.
5. **Milestone 5: MCP server.** Wrap everything as the eight tools with the Python MCP SDK; wire into Claude Code; run the headline demo.
6. **Milestone 6: benchmarks and write-up.** Run the benchmark suite below, record the demo, write the README and blog post.

The estimator's first version can be a formula; every finished run then becomes a data point for calibrating it.

Benchmarks and deliverables

The write-up should lead with numbers, not features. Run a fixed suite of about 10 jobs (3 model sizes × a few dataset sizes, plus 2 eval jobs) and measure:

Metric	Compared against	Why it matters
Total cost of the suite	A naive baseline: every job on one provider's A100	The core claim: cheaper with no manual work
VRAM estimate error	Peak memory actually observed	Shows the estimator is real, not a guess
Runtime and cost estimate error	Actual wall-clock and bill	Whether quotes can be trusted
Launch success rate	Jobs that OOM'd or failed to start	Honest reliability number
Orphaned GPU-hours	Target: zero	The guardrails work

Ship these with the repo:

- [ ] README with the architecture diagram and a 60-second quickstart
- [ ] 2–3 minute demo video of the headline request from Claude Code
- [ ] Blog post: the benchmark table, what the estimator got wrong, what you changed
- [ ] Tests for each adapter against recorded API responses, so CI never touches a real GPU

Risks and stretch goals

The biggest risk is the estimator: a bad VRAM guess means OOMs or overpaying, and that undercuts the whole cost story.

Risk	Mitigation
VRAM estimate too low, job OOMs	Add a safety margin; on OOM, auto-retry once on the next GPU size up and log the miss
Prices not available via API for every provider	Hand-maintained catalog with a last-updated date; flag stale quotes

Risk	Mitigation
GPU type sold out at launch time	`get_quotes` returns a ranked list; fall through to the next option
Providers' APIs differ more than expected	Keep the adapter interface small; push provider quirks into the adapter
Dev costs creep	$50 dev budget enforced by your own global cap

Stretch, only after the benchmarks are done:

- Spot/interruptible instances with automatic resume from the last checkpoint
- Learned estimator calibrated on your own run history
- A small read-only dashboard of runs and spend
- A fourth provider (Lambda or Vast.ai) to show the adapter pattern holds